Intro a Shell & Lab 1: `MyBash`

Presentación

Sistemas Operativos
FAMAF-UNC

Intro a Shell

Parte 1

Conceptos Básicos de Shell

Shell - ¿Qué es?

- **Definición general:**
  - Un shell es un **intérprete de línea de comandos**: un programa que recibe **instrucciones de texto** del usuario, las **interpreta** y las **ejecuta**.

```
Terminal
[marco@matrix shellscripting]$ ls
atpplayers.in  Bite_Size_Linux.pdf  README.md  respuestas.txt  superliga.in  weather_cordoba.in
[marco@matrix shellscripting]$ sort -rn --field-separator=' ' -k4 weather_cordoba.in | head -n 1
1986 1 25 348 397 282 10022 48 0
[marco@matrix shellscripting]$
```

En el Lab 0 usamos Bash, un shell en particular.

Shell - Funciones (1/3)

- **Funciones clave:**
  - **1) Interfaz Usuario-Sistema**: traduce comandos en llamadas al sistema (***syscalls***: el mecanismo que provee el sistema operativo para que los programas de **usuario** interactúen con el **kernel**). Ejemplos de Linux y xv6 (subset de Unix):
    - E.g., **Gestión de procesos**:
      - `fork()` -> Crea un proceso hijo.
      - `exec()` -> Ejecuta el proceso hijo (e.g., binario de `ls`).
      - `wait()` -> Espera a que termine el hijo.
    - E.g., **Archivos y directorios**: `open()`, `read()`, `write()`, `close()`, `chdir()`
    - E.g., **Comunicación entre procesos**: `pipe()`, `dup()`, `kill()`

Syscalls

Shell - Funciones (2/3)

Syscalls Linux vs. xv6 (subset educativo)

Categoría	Linux (POSIX/Unix-like)	xv6 (subset educativo)	Notas rápidas
Procesos	fork, vfork, execve (familia exec* vía libc), wait/ waitpid, exit, getpid, kill, nice, setpgid, setsid…	**fork, exec, wait, exit, getpid, kill**	**xv6 tiene lo mínimo para crear/reemplazar/esperar procesos**. No hay job control avanzado ni grupos de procesos.
Archivos/FS	open, close, read, write, lseek, stat/fstat, unlink, link, rename, chmod, chown, umask, mkdir, rmdir, symlink, readlink, mknod…	**open, close, read, write, fstat, link, unlink, mkdir, chdir, mknod**	**xv6 carece de permisos avanzados** (chmod, chown), symlinks, rename, etc.
Directorios	getcwd, chdir, mkdir, rmdir	**chdir, mkdir**	getcwd no aparece como syscall en xv6 (se puede reconstruir en userland).
IPC y descriptores	pipe, dup, dup2, socket, send/recv, shmget*…	**pipe, dup**	**xv6 no tiene sockets, shared memory, ni dup2** (se usa dup y cierres ordenados).
Señales	sigaction, sigprocmask, sigsuspend, alarm…	**kill (muy básico)**	**xv6 no implementa el modelo completo de señales** (no hay sigaction).
Memoria	brk/sbrk, mmap/munmap, mprotect…	**sbrk**	No hay mmap ni protección fina en xv6; **manejo de memoria simple**.
Tiempo	nanosleep, clock_gettime, gettimeofday (hist.)…	**sleep, uptime**	Temporización simple; sin clock de alta resolución.
Varios	ioctl, poll, select, epoll, ptrace, mount, umount, chroot, namespaces, cgroups…	-	Estos subsistemas **no existen en xv6**: es un SO docente, no de propósito general.

Shell - Funciones (3/3)

- **Funciones clave:**
  - 2) **Orquestador de Procesos**:
    - **Crea y controla procesos**: crear, terminar, esperar y coordinar programas.
      - e.g., `fork()` → `exec()` → `wait()`
    - **Comunica procesos**: conectar entrada/salida de programas.
      - e.g., **redirecciones** (`>`, `<`) y **pipes** (`|`).
  - 3) **Entorno de Ejecución**:
    - **Ejecución manual**: el usuario escribe comandos en el prompt.
    - **Ejecución automatizada**: a través de scripts `.sh` que secuencian tareas.

El shell no ejecuta programas por sí mismo: los organiza y delega al kernel mediante syscalls.

Shell - Arquitectura Interna (1/4)

Paradigma respetado por prácticamente todos los shells (bash, zsh, fish, dash)

- **Ciclo Principal** (**REPL**, o *Read–Evaluate–Print–Loop*):
  - **Read**: lee línea de comandos (desde *stdin*) y analiza (*parse*) la línea en estructura de datos interna
    - Divide en *tokens*, *comandos*, *pipes* → Tipo Abstracto de Datos (**TAD**).
  - **Evaluate**: ejecuta comandos:
    - **Internos**: se manejan directamente desde el mismo proceso.
    - **Externos**: `fork()` crea hijo, `exec()` en hijo, `wait()` en el padre.
  - **Print**: el shell deja que los procesos hijos escriban en stdout. El shell solo "imprime" el prompt (i.e., `juan@pc:~$`).
  - **Loop**: vuelve al inicio

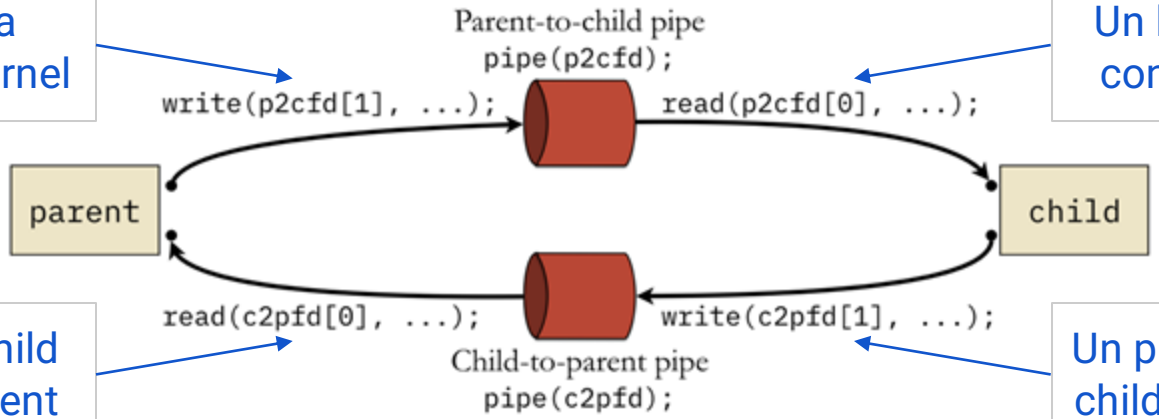
Diferente en e.g., Python que también usa REPL.

Shell - Arquitectura Interna (2/4)

- **Comunicación 1)** por **Pipes** → `pipe()` conecta procesos (*stdout* → *stdin*).
  - Un *pipe* es un **canal** unidireccional de comunicación. Sirve para que la **salida** estándar (stdout) de un proceso sea la **entrada** (stdin) de otro.

Un pipe es una estructura del kernel

Un buffer en mem. con dos extremos



p2c: parent to child
c2p: child to parent

Un pipe puede ser de child a child también

Shell - Arquitectura Interna (3/4)

- **Comunicación 2)** por **Duplicación de descriptores** → `dup()`, `dup2()`.
  - Cada proceso tiene descriptores de archivo:
    - `0` → stdin (entrada estándar, normalmente teclado).
    - `1` → stdout (salida estándar, normalmente terminal).
    - `2` → stderr (errores, normalmente terminal).
  - Cualquier *archivo*, *pipe* o *socket* abierto en un proceso se representa con un número de descriptor (`3`, `4`, …).
  - Se pueden **copiar** descriptores para **leer**/**escribir** datos usando *la misma entrada de tabla* de archivos en el kernel (mismo offset, o posición de read/write).
  - Permite que varios “lectores”/“escritores” avancen coordinadamente por el mismo archivo.

Shell - Arquitectura Interna (4/4)

- **Comunicación 3)** por **Redirección de Descriptores** → `open()`, `close()`, `dup2()`.
  - El shell puede **reemplazar** stdin/stdout/stderr con archivos reales
    - `ls > out.txt` → stdout (1) se asocia al archivo out.txt.
    - `sort < in.txt` → stdin (0) se asocia al archivo in.txt.
    - `grep foo 2> err.txt` → stderr (2) se asocia a err.txt.
  - Mecánica:
    - `open("out.txt", O_WRONLY|O_CREAT)` → devuelve un FD (ej. 3).
    - `dup2(3, STDOUT_FILENO)` → ahora stdout (1) apunta al archivo.
    - `close(3)` → se libera el descriptor extra.
    - Antes de redirección: stdout (1) → terminal
    - Después de redirección: stdout (1) → `out.txt`

Shell Bash

Bash es descendiente de sh (Bourne Shell), creado en los ’70 para Unix.

- Bash (Bourne Again SHell) es un **shell avanzado** de GNU/Linux con:
  - **Comandos internos** (*builtins*) como `cd`, `export`, `history`.
  - **Comandos externos** vía `execvp()` tras un `fork()`.
  - **Redirecciones** (`>`, `<`) y pipes (`|`).
  - **Ejecución en segundo plano** (`&`) o *background*.
  - **Scripting completo** (`if`, `for`, `while`, funciones y variables).
  - **Historial y edición interactiva** (↑, ↓, búsqueda incremental con Ctrl+R.)
  - **Configuración personalizable** en ~/.bashrc y ~/.bash_profile.

Lab 1: `MyBash`

Parte 2

Presentación Lab 1

Shell MyBash

- **Lab 0**: Usamos `Bash`
  - Desde la perspectiva del usuario:
    - Comandos básicos, scripts `.sh`, automatización de tareas.
- **Lab 1**: Implementamos `MyBash`:
  - Un Shell reducido, minimalista.

Shell MyBash - Elementos Básicos

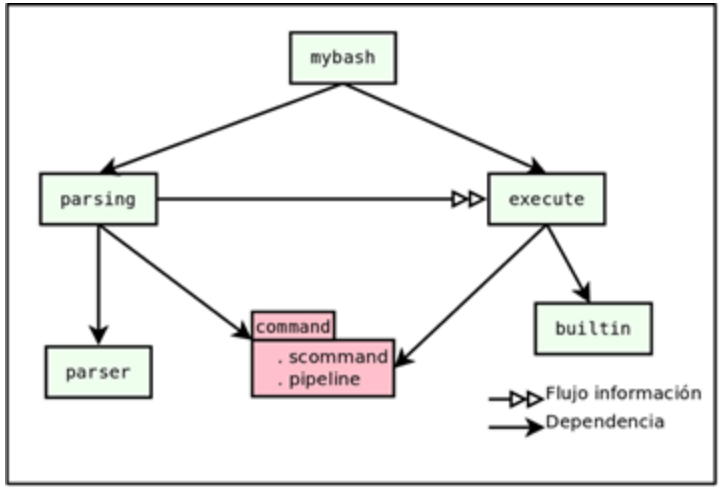
- **Comandos internos** (*builtin*):
  - `cd`: cambiar directorio con chdir().
  - `help`: mostrar ayuda.
  - `exit`: terminar el shell.
- **Comandos externos**:
  - Identificar programa → `fork()` → `execvp()` → `wait()`.
- **Redirecciones**:
  - Entrada (`<`) y salida (`>`).
- **Pipes** (`|`):
  - Conectar *stdout* de un comando con *stdin* del siguiente.
- **Ejecución en background**:
  - Detectar `&` y no bloquear el shell.

Ejemplo de comando que podremos correr con `MyShell`:

```
ls -l | grep ".c" > archivos_c.txt &
```

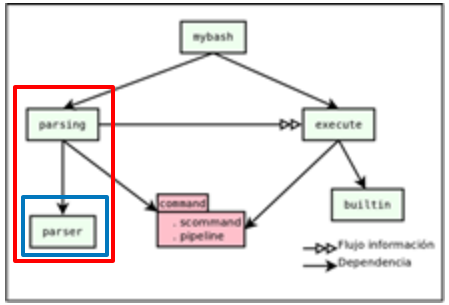
Shell MyBash - Módulos

- **mybash**: módulo principal
- **command**: módulo con las definiciones de los TADs.
- **parsing**: módulo de procesamiento de la entrada del usuario (el **parser** implementa el TAD parser).
- **builtin**: módulo que implementa los **comandos internos**.



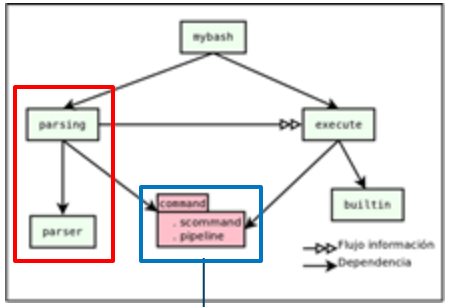
- **execute**: módulo ejecutor de comandos, administra las llamadas al sistema operativo

Módulo Parsing



- **Parser** (dado por la cátedra)
  - Permite acceder al **TAD** (Tipo Abstracto de Datos) central. El TAD:
    - Define qué **operaciones** se pueden hacer sobre un conjunto de datos.
    - Permite trabajar con un **modelo abstracto** sin preocuparse de cómo está implementado.
  - Interfaz **provista** en `parser.h`, y sus respectivas implementaciones en `parser.o` y `lexer.o` (para arquitecturas i386 y x86_64).
  - **Comportamiento**: Recibe la entrada (*stdin*) y devuelve una estructura de datos lista para ejecutar. Las <u>estructuras principales</u> son: `scommand` y `pipeline`…

Módulo Parsing



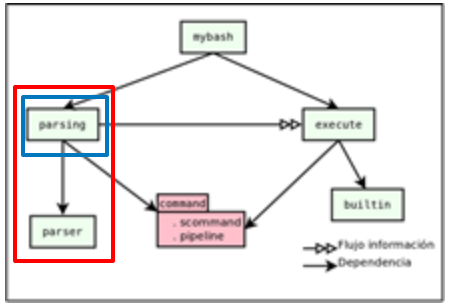
- **Parser** (dado por la cátedra)
  - Estructuras principales del TAD son:

    Definidas en `command.h` a implementar en `command.c` por ustedes
    - `scommand` → representa un **comando** con sus argumentos.
      - **Nombre del programa** (ej. `ls`) y sus **argumentos** (ej. `["-l", "/home"]`)
      - **Redirecciones** (`>`, `<`) y ejecución en **background** (`&`).
    - `pipeline` → representa una **secuencia** de comandos conectados por |.
      - Cada comando recibe la **salida** del anterior como **entrada** (ej. `ls -l | grep ".c"`)

```
[ scommand ]
  ├── Nombre: "ls"
  ├── Args: ["-l", "ej1.c"]
  ├── Redirección salida: "out.txt"
  └── Redirección entrada: NULL
```

```
// Pipe
[ scommand: ls -l ]  →
[ scommand: grep ".c" ]  →
[ scommand: wc -l ]
```

Módulo Parsing



- **Parsing** (a completar en el lab)
  - Recorre *stdin* de manera lineal y construye una instancia del TAD pipeline.
  - **Resultado**: una representación abstracta de los comandos ingresados.

```
cat in.txt | grep "error" > out.txt &
```

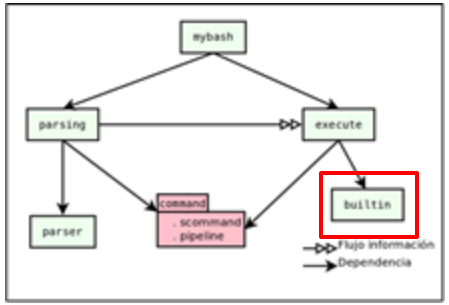
```
["cat","in.txt","|","grep","error",">","out.txt","&"]
```

```
pipeline {
  cmds = [
    scommand{ cmd="cat",  args=["in.txt"], in=NULL, out=NULL },
    scommand{ cmd="grep", args=["error"],  in=NULL, out="out.txt" }
  ],
  background = true}
```

Stdin → Token

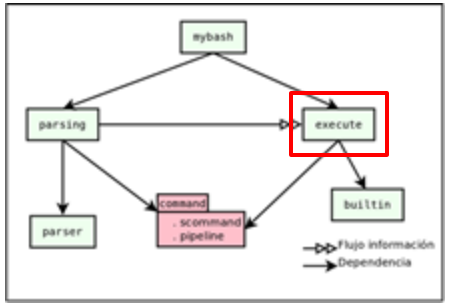
En **parsing** se construye estas “estructuras abstractas”

Módulo Builtin



- Gestiona los **comandos internos** del shell:
  - Detecta si la entrada corresponde a un comando interno y la ejecuta.
- **Comandos** implementados
  - `cd` → cambia el directorio de trabajo (usa syscall `chdir()`).
  - `help` → imprime por stdout:
    - Nombre del shell y autores.
    - Lista de comandos internos disponibles con breve descripción.
  - `exit` → termina el shell de manera limpia (libera recursos antes de salir).
- **Notas**
  - Los builtins se ejecutan dentro del proceso del shell, no en un hijo.
  - La arquitectura debe permitir agregar fácilmente nuevos comandos internos.

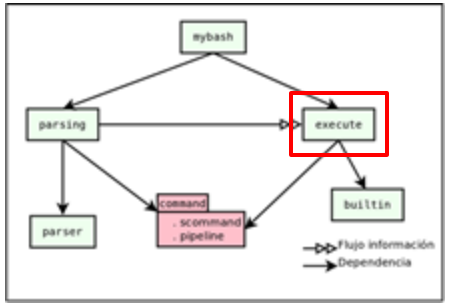
Módulo Execute: SysCalls (1/5)



- `chdir(path)` → Cambia el directorio de trabajo
  - Nota: `cd` debe ejecutarse en el mismo proceso (no en un hijo), ya que el directorio de trabajo es un atributo del proceso.
  - Ejemplo de uso:

```
if (chdir("/home/juan") < 0) {
    perror("chdir");
}
```

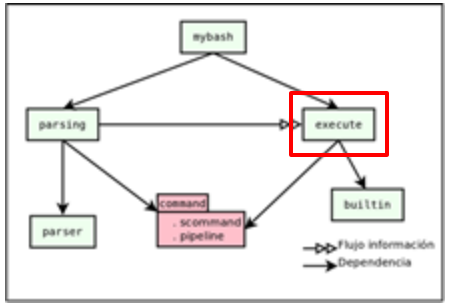
Módulo Execute: SysCalls (2/5)



- `fork()` → Duplica el proceso (padre e hijo).
- `execvp()` → Reemplaza la memoria del hijo con el nuevo programa.
- `wait()` → El padre espera a que el hijo termine.
- Ejemplo de uso para comando externo `ls -l`:

```
if (fork() == 0) {
    char *argv[] = {"ls", "-l", NULL};
    execvp(argv[0], argv);
    perror("execvp");
    exit(1);
} else {
    wait(NULL); // padre espera
}
```

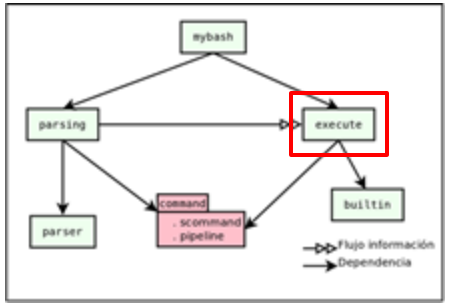
Módulo Execute: SysCalls (3/5)



- `fork()` → Duplica el proceso (padre e hijo).
- `execvp()` → Reemplaza la memoria del hijo con el nuevo programa.
- Nota: NO llamamos a `wait()` para ejecución en *background*.
- Ejemplo de uso para comando externo `ls -l`:

```
if (fork() == 0) {
    char *argv[] = {"xeyes", NULL};
    execvp(argv[0], argv);
    perror("execvp");   // solo si falla
    exit(1);
}
// padre NO llama a wait(), vuelve al prompt
```

Módulo Execute: SysCalls (4/5)



- `open()/close()` → Abre archivo y cierra descriptores.
- `dup()/dup2()` → Redirige la entrada/salida estándar.
- Ejemplo de uso para comando externo `wc -w < in.txt > out.txt`:

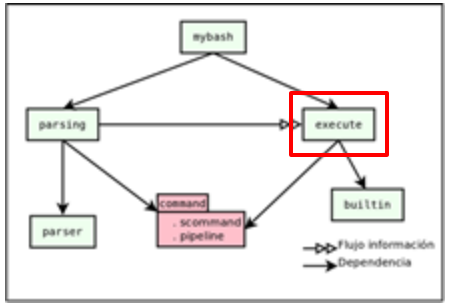
```
int in = open("in.txt", ...);
int out = open("out.txt", ...);
dup2(in, STDIN_FILENO);   // redirige stdin al archivo in.txt
dup2(out, STDOUT_FILENO); // redirige stdout al archivo out.txt

close(in); close(out);
char *argv[] = {"ls", "-l", NULL};
execvp(argv[0], argv);
```

Tarea: cómo queda out.txt si in.txt es “hola mundo”?

Tarea: cual es la diferencia entre dup() y dup2()?

Módulo Execute: SysCalls (5/5)



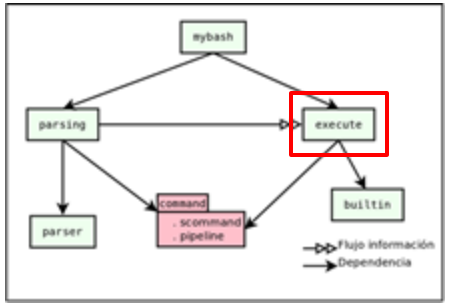
- `pipe()` → Crea descriptores: lectura (`p[0]`) y escritura (`p[1]`).
- Ejemplo de uso para comando externo `ls | wc -l`:

```
int p[2]; pipe(p); // p[0] → lectura, p[1] → escritura

// Fork 1
dup2(p[1], STDOUT_FILENO); // Redirige la salida std a el pipe de escritura
char *argv[] = {"ls", NULL}; execvp(argv[0], argv);

// Fork 2
dup2(p[0], STDIN_FILENO); // Redirige la entrada std para leer del pipe.
char *argv[] = {"wc", "-l", NULL}; execvp(argv[0], argv);
```

Módulo Execute: Ejemplos



Entrada	SysCalls relacionadas	Comentario
`cd ../..`	`chdir()`	Comando interno, llama la *syscall* de cambio de dir.
`gzip Lab1G04.tar`	`fork(); execvp(); wait()`	Ejecutar el comando y el padre espera.
`xeyes &`	`fork(); execvp()`	Un comando simple sin redirectores y sin espera.
`ls -l ej1.c > out < in`	`fork(); open(); close(); dup(); execvp(); wait()`	Redirige tanto la entrada y salida. El shell padre espera.
`ls | wc -l`	`pipe(); fork(); open(); close(); dup(); execvp(); wait()`	Sin ejecución en 2do plano, dos comandos simples conectados por un pipeline.

Manejo de Strings (1/2)

- En C no existen cadenas como tipo nativo. Una **string** es un puntero a char con un carácter nulo (`\0`) al final.
  - `char saludo[] = "Hola"; // ['H','o','l','a','\0']`
- Para trabajar con strings, usamos la librería estándar `<string.h>`.
  - `strlen(s)` → longitud.
  - `strcpy(dst, src)` → copia.
  - `strcat(dst, src)` → concatenar.
  - `strcmp(s1, s2)` → comparar.

Manejo de Strings (2/2)

- Además de `<string.h>`, el laboratorio provee `strextra.h`.
- Allí se declara la función `strmerge` que crea una **nueva cadena** con el contenido de s1 seguido de s2:
  - `char *strmerge(const char *s1, const char *s2);`
- Evita que cada estudiante implemente manualmente concatenación con `malloc` + `strcpy`.

Diferencia entre **`strcat`** (en `string.h`) y **`strmerge`** (en `strextra.h`) :

- **`strcat`**: Concatena in-place en el buffer de destino, lo que implica riesgo de overflow si no hay espacio suficiente.
- **`strmerge`**: Crea una nueva cadena dinámica con la concatenación. Seguro, pero requiere liberar memoria.

Tests

- Se incluyen baterías de test para ayudarles a debuguear
- Pruebas de command.c:
  - `$ make test-command`
- Pruebas de parsing.c:
  - `$ make test-parsing`
- Pruebas para todos los módulos juntos:
  - `$ make test`
- Pruebas de manejo de memoria en los módulos:
  - `$ make memtest`

La entrega tiene que pasar todos los tests para aprobar.

Enunciado y Entrega

- El **enunciado** ayuda mucho con la tarea de implementación:
  - Leanlo **detenidamente** y sigan los consejos que se les ofrece.
- Elementos de **entrega**:
  - Código fuente: main branch del **Bitbucket** asignado por la cátedra.
  - Readme: en la raíz del main branch.
  - Video: máximo de **7 minutos** con todos los miembros del grupo.

Intro a Shell & Lab 1: `MyBash`

Presentación

Sistemas Operativos
FAMAF-UNC